

Layer-based Architecture

SWEN1 | BIF3

Table of Contents

- Layered Architecture



Layered Architecture

Layered Application

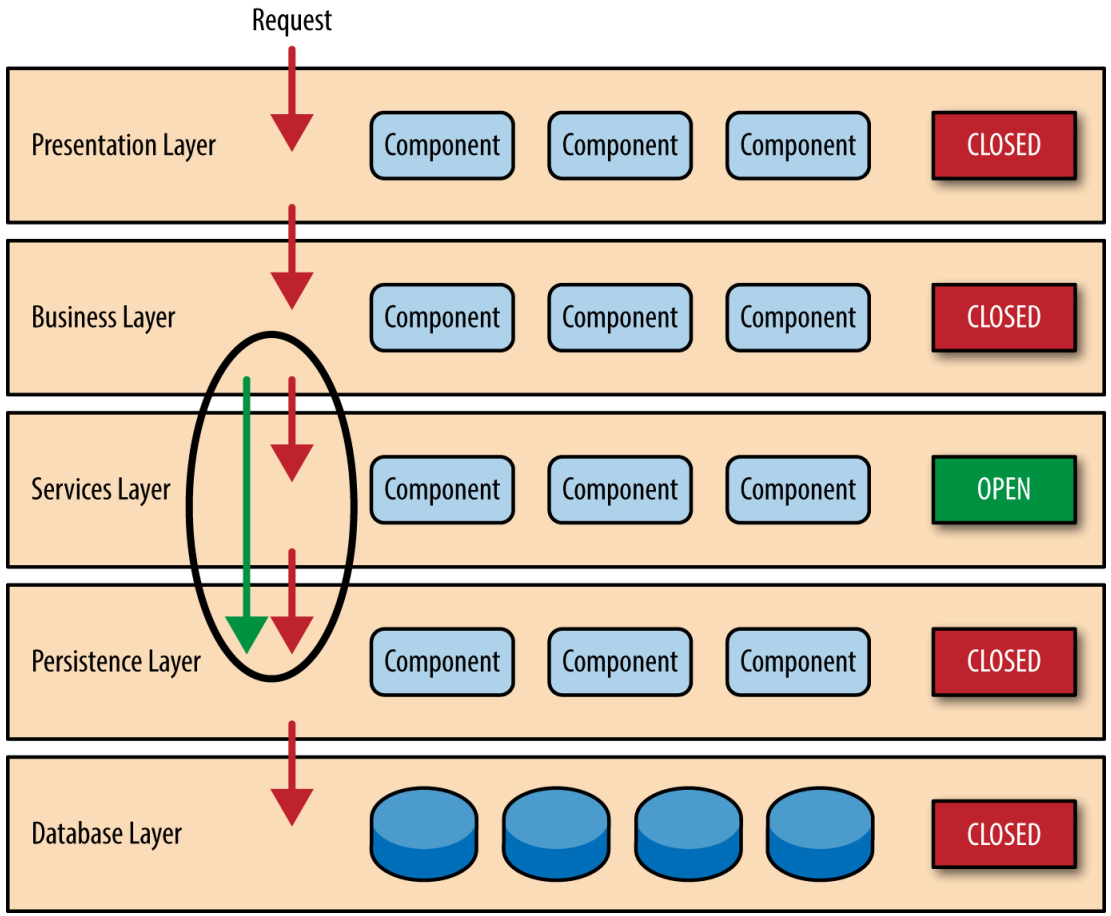
...the overall structure for applications in terms of the logical grouping of components into separate layers that communicate with each other and with other clients and applications.

It is important to understand the distinction between layers and tiers:

- **Layers** describe the logical groupings of the functionality and components in an application,
- **Tiers** describe the physical distribution of the functionality and components on separate servers, computers, networks, or remote locations.

Layered Architectural Style

- Grouping of related functionality into distinct layers, stacked vertically on top of each other
- Functionality in each layer related by a common role or responsibility (Data access, ...)
- Communication between layers is explicit & loosely coupled
- Only the dependencies on the N-1 Layer allowed

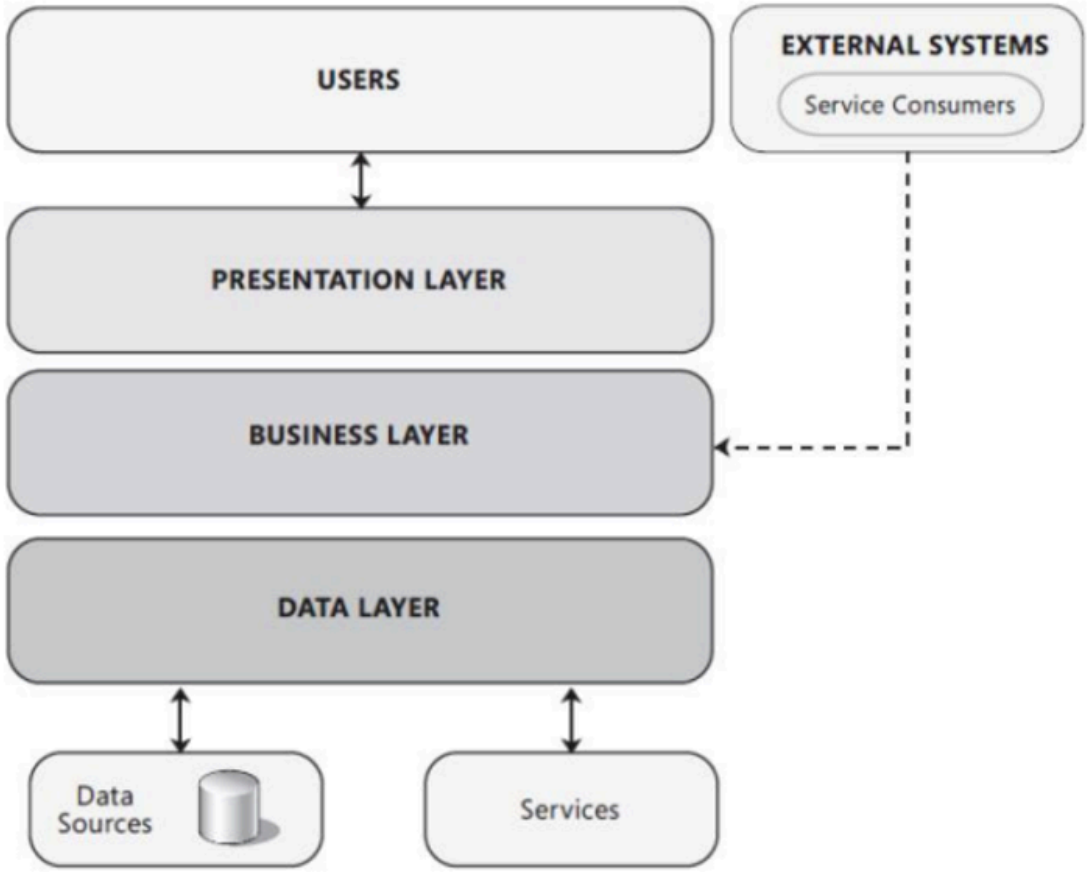


Common Principles

- Abstraction
- Encapsulation
- Clearly defined functional layers
- Reusable
- Loose coupling

3-Layer Application

At the highest and most abstract level, the **logical architecture** view of any system can be considered as a set of cooperating components grouped into layers.

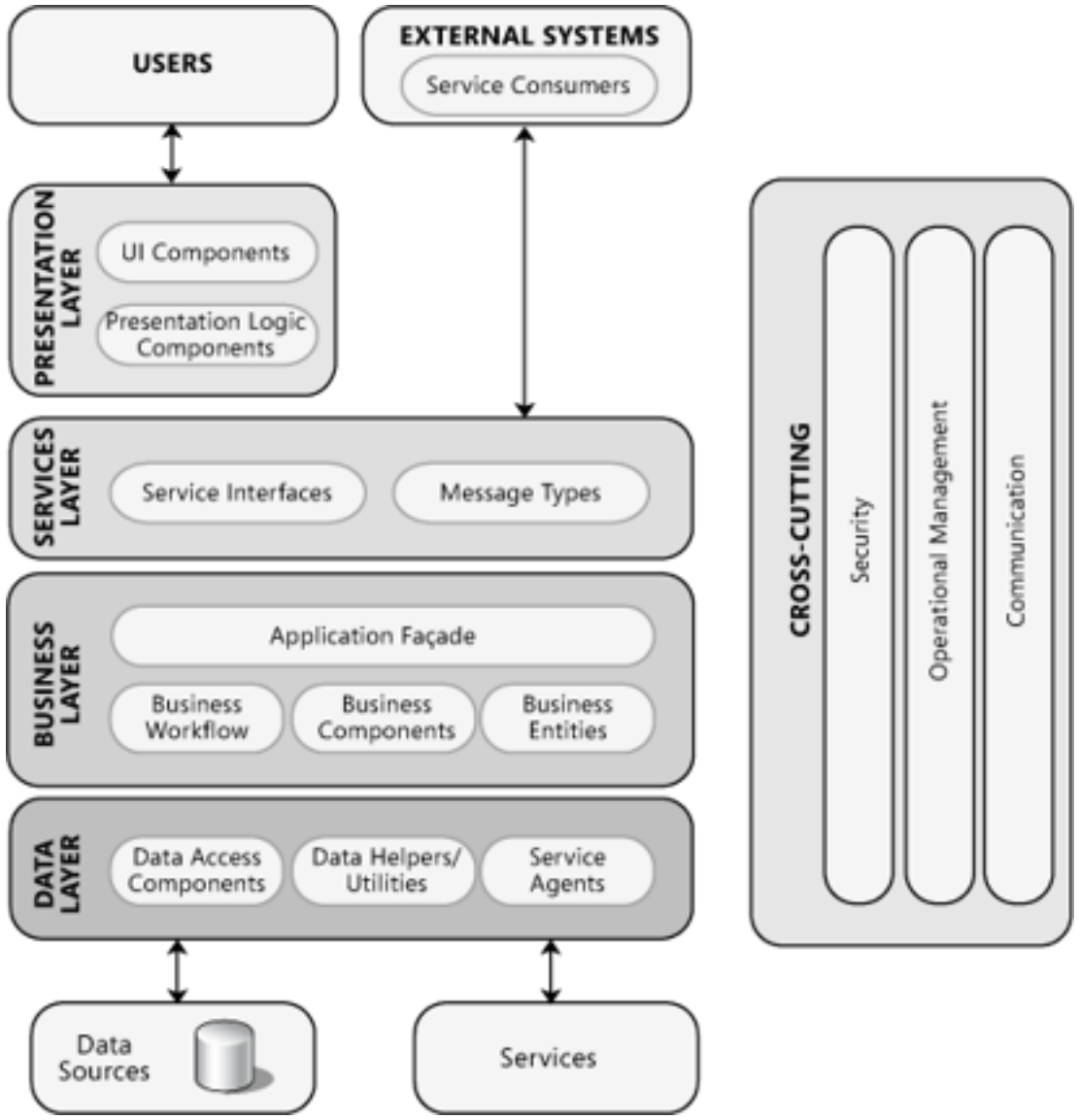


More detailed view

including:

- Services-Layer
- Cross-Cutting Concepts

When an application must provide services to other applications, as well as implementing features to support clients directly, a common approach is to use a services layer that exposes the business functionality of the application.



Presentation Layer

This layer contains the **user oriented functionality** responsible for managing user interaction with the system, and generally consists of components that provide a common **bridge into the core business logic** encapsulated in the business layer.

- UI Components: visual elements (="Controls")
- Presentation Logic Components: UI behavior, flow validation,...

Use appropriate patterns:

- MVVM: Model-View-ViewModel
- MVP: Model-View-Presenter
- MVC: Model-View-Controller

Presentation Layer Technologies

Technologies:

- Java: JavaFX, Swing, AWT
- .Net: WPF (Windows-Presentation-Foundation), WinForms
- HTML / ASP.Net / JSP,..
- MFC, QT, GTK

Challenges

- Validation
- Navigation
- Exception Management
- Composition
- Communication
- Caching

Services Layer

- Important separation: it hides the internal implementation
- Service contracts
- Message types
- Translator components

Protocols:

- REST, SOAP/WSDL, XML-RPC,...

Frameworks/Infrastructure:

- Windows Communication Foundation / IIS
- Spring/Spring Boot

Services and Layers

From a high level perspective, a **service-based solution** can be seen as being composed of multiple services.

Conceptually, the services can be seen as **components of the overall solution** . However, internally, each service is made up of software components, just like any other application, and these components can be logically grouped into presentation, business, and data layers.

Other applications can make use of the services without being aware of the way they are implemented.

Business Layer

This layer implements the **core functionality** of the system, and encapsulates the relevant business logic. It generally consists of components, some of which may expose service interfaces that other callers can use.

Application facade:

- Simplified interface
- Hides complexity
- Combines multiple operations / components
- Just forwards calls, does security checks, …

Business Logic components:

- **Business Workflow components**: processes, orchestration, long-running
- **Business components**: encapsulate logic, calculations, validation
- **Business Entity components**: store data, also validation & logic

Data Layer

This layer provides **access to data** hosted within the boundaries of the system, and data exposed by other networked systems; perhaps accessed through services. The data layer exposes generic **interfaces** that the components in the business layer can consume.
(see also: Anti-Corruption Layer)

- Access Data Sources of different kinds: relational, XML, services
- **Data Access Components**: encapsulate data source specific code, grouped by source-type
- **Service Agents**: encapsulate access to service proxies